UNIVERSITY SCHOOL OF AUTOMATION AND ROBOTICS

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

EAST DELHI CAMPUS , SURAJMAL VIHAR , DELHI 110034

PROJECT REPORT

ON

IOT SMART ROOM MONITORING AND SECURITY DEVICE

Submitted By-:

NAME **LAKSHYA**

ENROLL NO. **09519051724**

BRANCH **IIOT B2 A**

Under The Supervision Of

Dr. Khyati Chopra USAR

Abstract

This project presents the development of an **IoT-based Smart Room Environmental Monitoring and Security System** using the **ESP32-CAM** module. The system integrates multiple sensors — **DHT22 for temperature and humidity, MQ-2** for **smoke/gas detection**, **flame sensor for fire detection**, and **PIR sensor for motion detection — to continuously monitor the room environment and security conditions**.

The ESP32-CAM captures live images upon **detection of any anomaly** (gas, flame, or motion) and instantly sends emergency alerts with photos and **live stream** links to the user **via Telegram**. Real-time sensor data is displayed on a **Blynk dashboard** for **remote monitoring**. The system uses a centralized 5V power rail with stabilization capacitor to ensure reliable operation during Wi-Fi and camera activity.

By combining edge processing, cloud connectivity, and visual confirmation through the built-in camera, the proposed system provides an affordable, compact, and highly responsive solution for smart home safety and security. It significantly reduces response time to **environmental hazards** and **unauthorized intrusions** while maintaining **low cost and easy installation**.

Introduction

In today's smart homes and offices, continuous monitoring of **environmental parameters and security threats** has become essential for **safety and comfort**. Traditional systems often rely on separate devices for **temperature, smoke, fire, and motion detection,** leading to high cost, complexity, and delayed response.

Major risks in indoor spaces include **fire outbreaks, gas leakage,** and **unauthorized entry. Early detection** of these events is critical to prevent damage and ensure occupant safety. However, most low-cost solutions lack integrated visual confirmation and instant cloud alerting.

The proposed IoT-based Smart Room Monitoring System **addresses these challenges by using a single ESP32-CAM module that combines environmental sensing, motion security, camera-based verification, and cloud connectivity.** The system automatically detects anomalies, captures images, and sends real-time alerts with photos and live stream links to the user's mobile via Telegram, while displaying all data on a Blynk dashboard.

This low-cost, compact solution can be easily installed in any room, hostel, office, or laboratory, providing reliable **24×7 monitoring and rapid response capability.**

Objectives

- To develop a compact IoT-based system **for real-time monitoring** of temperature, humidity, smoke/gas, fire, and motion in a room.
- To **detect environmental hazards** (gas, smoke, flame) and security threats (intrusion) using multiple sensors.
- To **automatically capture images** and send **instant alerts** with photos and **live stream links via Telegram** upon anomaly detection.
- To display all sensor data on **a user-friendly Blynk dashboard for remote monitoring**.
- To ensure boot safety and power stability using proper GPIO configuration and capacitor.
- **To create a low-cost, easy-to-install solution using ESP32-CAM that can be deployed in homes, offices, or laboratories.**
- To minimize false alarms through debounce logic and provide reliable emergency response**.**

Literature Review

Recent advancements in the field of the Internet of Things (IoT) have enabled the development of intelligent monitoring and security systems that integrate sensing, communication, and automation into a single platform. Smart room monitoring systems have gained significant attention due to their ability to provide real-time environmental analysis and enhance safety through automated alerts.

Earlier research primarily focused on standalone environmental monitoring systems using microcontrollers such as Arduino Uno and basic sensors like temperature and gas sensors. These systems were limited in functionality, often lacking remote access and real-time alert capabilities. Communication was commonly achieved through GSM modules, which, although effective, introduced limitations such as high cost, slower response time, and dependency on cellular networks.

With the emergence of advanced microcontrollers like the ESP32, researchers began integrating Wi-Fi-based communication into monitoring systems. This shift enabled faster data transmission, cloud connectivity, and real-time visualization through mobile applications. Several studies have demonstrated the use of ESP32 for smart home automation, environmental monitoring, and security applications due to its built-in Wi-Fi, low power consumption, and high processing capability.

In the domain of safety monitoring, gas detection systems using MQ-series sensors (such as MQ-2 gas sensor) have been widely used to detect hazardous gases like LPG and smoke. Similarly, flame sensors based on infrared detection have been utilized for early fire detection. Motion detection using PIR (Passive Infrared) sensors has also been extensively studied for intrusion detection and smart security systems.

However, most existing systems treat these functionalities independently. For example, some systems focus only on gas leakage detection, while others concentrate on motion detection or temperature monitoring. Very few implementations integrate multiple sensing capabilities into a unified system that can simultaneously monitor environmental conditions and security threats.

Recent research trends show integration of IoT platforms such as Blynk for real-time monitoring dashboards and cloud-based data visualization. These platforms allow users to remotely access sensor data, analyze historical trends, and control

devices through mobile applications. Additionally, messaging platforms like Telegram have been incorporated for instant alert notifications, enabling faster response during emergencies.

Another important advancement is the integration of camera modules for visual monitoring. Systems based on ESP32-CAM have enabled image capture and live streaming capabilities, providing visual verification of events such as motion detection or fire incidents. Despite this, many implementations either lack efficient trigger-based alert mechanisms or continuously stream data, leading to higher power consumption and reduced efficiency.

The proposed system improves upon these limitations by integrating multiple sensors—temperature (DHT22), gas (MQ-2), flame, and motion (PIR)—with an ESP32-CAM module to create a unified smart monitoring solution. Unlike traditional systems, it employs event-based triggering, where alerts, image capture, and live streaming links are generated only when an abnormal condition is detected. This approach optimizes power usage and reduces unnecessary data transmission.

Furthermore, the system combines real-time monitoring through Blynk with instant alerting via Telegram, ensuring both continuous observation and immediate response capability. The inclusion of a web server for live streaming enhances situational awareness, making the system more reliable and practical for real-world applications.

Overall, the literature indicates that while significant progress has been made in IoT-based monitoring systems, there remains a gap in integrating environmental sensing, security monitoring, real-time visualization, and intelligent alert mechanisms into a single, cost-effective solution. The proposed ESP32-CAM-based smart room monitoring system addresses this gap by providing a comprehensive, scalable, and efficient IoT-based safety solution.

HARDWARE REQUIREMENTS

1. ESP32-CAM (Main Controller + Camera Unit)

Function:

- Acts as the core processing unit of the system
- Connects to WiFi network
- Captures images during alert conditions
- Hosts a web server for live video streaming
- Sends alerts via Telegram and updates data to Blynk dashboard

2. DHT22 Sensor (Temperature & Humidity Sensor)

Function:

- Measures ambient temperature and humidity
- Sends digital data to ESP32-CAM
- Used for environmental monitoring and real-time display on Blynk

3. MQ-2 Gas Sensor (Smoke/Gas Detection)

Function:

- Detects LPG, smoke, and combustible gases
- Provides digital output when gas level exceeds threshold
- Helps in fire hazard detection and safety monitoring

4. Flame Sensor Module (IR-Based Fire Detection)

Function:

- Detects infrared radiation emitted by flames
- Outputs digital signal when fire is detected
- Enables early fire detection and alert triggering

5. PIR Motion Sensor (HC-SR501)

Function:

- Detects human motion using infrared radiation

- Outputs HIGH signal when motion is detected
- Used for intrusion/security monitoring

6. 10kΩ Resistor (Pull-Down Resistor)

Function:

- Connected between GPIO 12 and GND
- Ensures ESP32 boots correctly by keeping GPIO 12 LOW
- Prevents boot failure due to floating pin state

7. Electrolytic Capacitor (1000µF Recommended)

Function:

- Connected between 5V and GND
- Stabilizes power supply
- Prevents brownout resets during WiFi transmission and camera operation

8. Breadboard / Zero PCB

Function:

- Used to assemble and connect all components
- Provides organized layout for circuit connections
- Can be replaced by soldered PCB for permanent setup

9. Jumper Wires / Connecting Wires

Function:

- Used for electrical connections between components
- Ensures proper signal and power flow

10. 5V 2A Power Adapter (SMPS / Mobile Charger)

Function:

- Supplies stable power to ESP32-CAM and sensors
- Required to handle current spikes during camera and WiFi operation

11. ESP32-CAM-MB Programmer (Optional)

Function:

- Used for uploading code to ESP32-CAM via USB
- Provides easy interface for programming

12. Enclosure / Project Box

Function:

- Houses all components securely
- Protects circuit from dust and damage
- Provides proper mounting for sensors and camera

SOFTWARE REQUIREMENTS

1. Arduino IDE

Function:

- Used to write, compile, and upload code to ESP32-CAM
- Provides development environment for embedded programming

2. Embedded C++ (Arduino Language)

Function:

- Used to implement sensor logic, alert system, and communication
- Controls hardware and processes sensor data

3. ESP32 Board Package (Espressif Systems)

Function:

- Provides necessary drivers and libraries for ESP32
- Enables WiFi, camera, and GPIO functionalities

4. Blynk IoT Platform

Function:

- Provides real-time dashboard for monitoring sensor data
- Displays temperature, humidity, gas, flame, and motion status
- Supports historical data visualization

5. Telegram Bot API

Function:

- Sends real-time alerts, images, and live stream link to user
- Enables remote notification system

6. WiFi Network (Internet Connectivity)

Function:

- Enables communication between ESP32 and cloud services
- Required for Telegram alerts and Blynk updates

7. Web Server (ESP32 Internal Server)

Function:

- Hosts live video streaming
- Provides IP-based access to camera feed

8. Serial Monitor (Arduino IDE)

Function:

- Used for debugging and monitoring system behavior
- Displays IP address, sensor values, and errors

Circuit Description

1. Main Unit (ESP32-CAM Based System)

The system is built around the ESP32-CAM, which acts as the central processing unit. It integrates sensing, image capture, Wi-Fi communication, and alert mechanisms into a single platform.

The ESP32-CAM continuously monitors sensor inputs, processes data, and triggers alerts (Telegram + Blynk) when abnormal conditions are detected.

a) Temperature & Humidity Sensor (DHT22)

The DHT22 sensor is connected to GPIO 14 of the ESP32-CAM.

- It measures ambient temperature and humidity in real time.
- The sensor sends digital data using a single-wire protocol.
- Data is processed and transmitted to the Blynk dashboard for live monitoring and historical analysis.

Working:
The sensor periodically sends environmental readings, which are displayed on the IoT dashboard.

b) Gas Sensor (MQ-2)

The MQ-2 gas sensor is connected to GPIO 13 (digital output).

- Detects LPG, smoke, and combustible gases.
- Outputs LOW signal when gas concentration exceeds threshold.
- Used for fire hazard and leakage detection.

Working:
When gas is detected, the sensor triggers an alert signal to ESP32, which initiates the alert system.

c) Flame Sensor (IR-Based Fire Detection)

The flame sensor is connected to GPIO 12.

- Detects infrared radiation emitted by flames.
- Provides digital output (LOW when flame detected).

Important Circuit Note:
A **10kΩ pull-down resistor** is connected between GPIO 12 and GND to ensure proper boot of ESP32-CAM.

Working:
When flame is detected, the sensor triggers an alert condition immediately.

d) PIR Motion Sensor (HC-SR501)

The PIR sensor is connected to GPIO 2.

- Detects human motion using infrared radiation.
- Outputs HIGH signal when motion is detected.
- Used for security and intrusion detection.

Working:
When motion is detected, ESP32 activates alert system and captures image.

e) Built-in LED Indicator

The ESP32-CAM has an internal LED connected to GPIO 33.

- Used for visual alert indication.
- Turns ON during alert condition.

f) Camera Module (OV2640)

The camera is integrated within ESP32-CAM.

- Captures image during alert conditions.
- Supports live streaming through web server.

Working:
When any sensor triggers:

- Image is captured
- Sent via Telegram
- Live stream link is generated

g) Power Supply System

The system is powered using a **5V 2A adapter**.

- 5V is supplied to ESP32-CAM (5V pin)

- All sensors are powered from same 5V rail
- Common ground is maintained

Stability Components:

- **Electrolytic Capacitor (1000µF)** between 5V and GND
  → prevents voltage drops and brownout
- Ensures stable operation during WiFi + camera usage

2. Communication & IoT Integration

a) Wi-Fi Communication

The ESP32-CAM connects to WiFi network.

- Enables cloud communication
- Sends alerts and updates in real-time

b) Blynk IoT Dashboard

Using Blynk:

- Displays temperature and humidity
- Shows sensor status (Gas, Flame, Motion)
- Provides real-time monitoring
- Stores historical data

c) Telegram Alert System

Using Telegram Bot API:

- Sends alert message when abnormal condition occurs
- Sends captured image
- Sends live streaming IP link

Example Alert Flow:

- Motion / Gas / Flame detected
- Alert message sent
- Photo sent
- Live stream link shared

3. System Operation Flow

1. System powers ON
2. ESP32 connects to WiFi
3. Sensors start monitoring environment
4. Data sent continuously to Blynk

When abnormal condition occurs:

- Sensor triggers ESP32
- LED turns ON
- Camera captures image
- Telegram alert is sent
- Live stream link generated

Component	Pin Name	Connect To (ESP32-CAM / Rail)	Type	Important Notes
ESP32-CAM	5V	5V Power Supply	Power	Use 5V 2A adapter
	GND	GND Rail	Power	Common ground for all
	GPIO 14	DHT22 DATA	Digital	Safe pin
	GPIO 13	MQ-2 DO	Digital	No boot issue
	GPIO 12	Flame DO	Digital	Needs pull-down
	GPIO 2	PIR OUT	Digital	Boot-safe if LOW
	GPIO 33	Built-in LED	Output	LOW = ON

"All components share a common ground reference and are powered from a stable 5V rail, ensuring proper signal integrity and system stability."

Breadboard Wiring

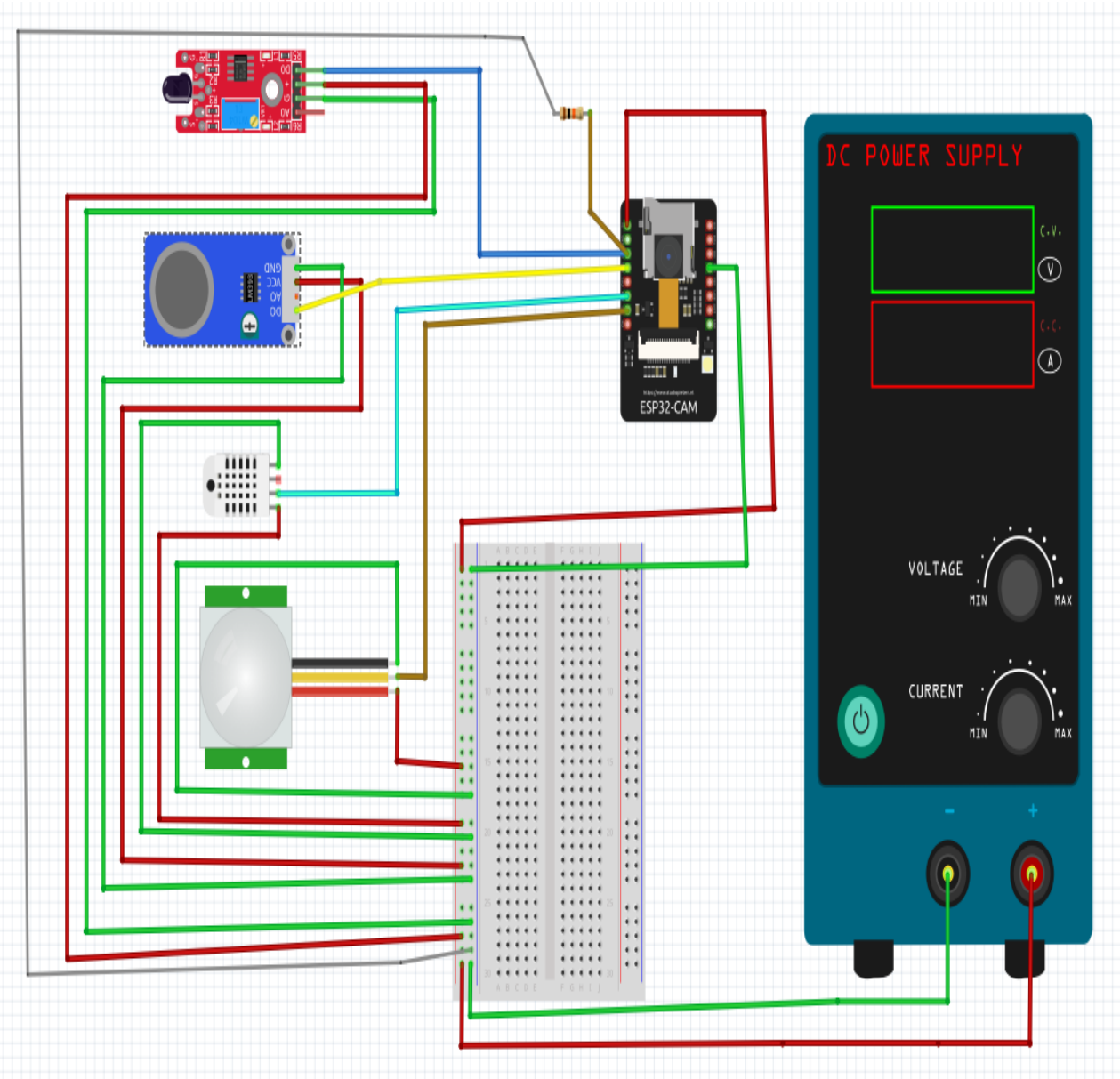

Fig.1

The breadboard setup is used for testing and prototyping the system. All components are connected using jumper wires with a common 5V and GND rail. It allows easy debugging and modification before final implementation.

Fritzing Software is Used For Breadboard Wiring , Schematic Diagram and PCB Design

Schematic Diagram

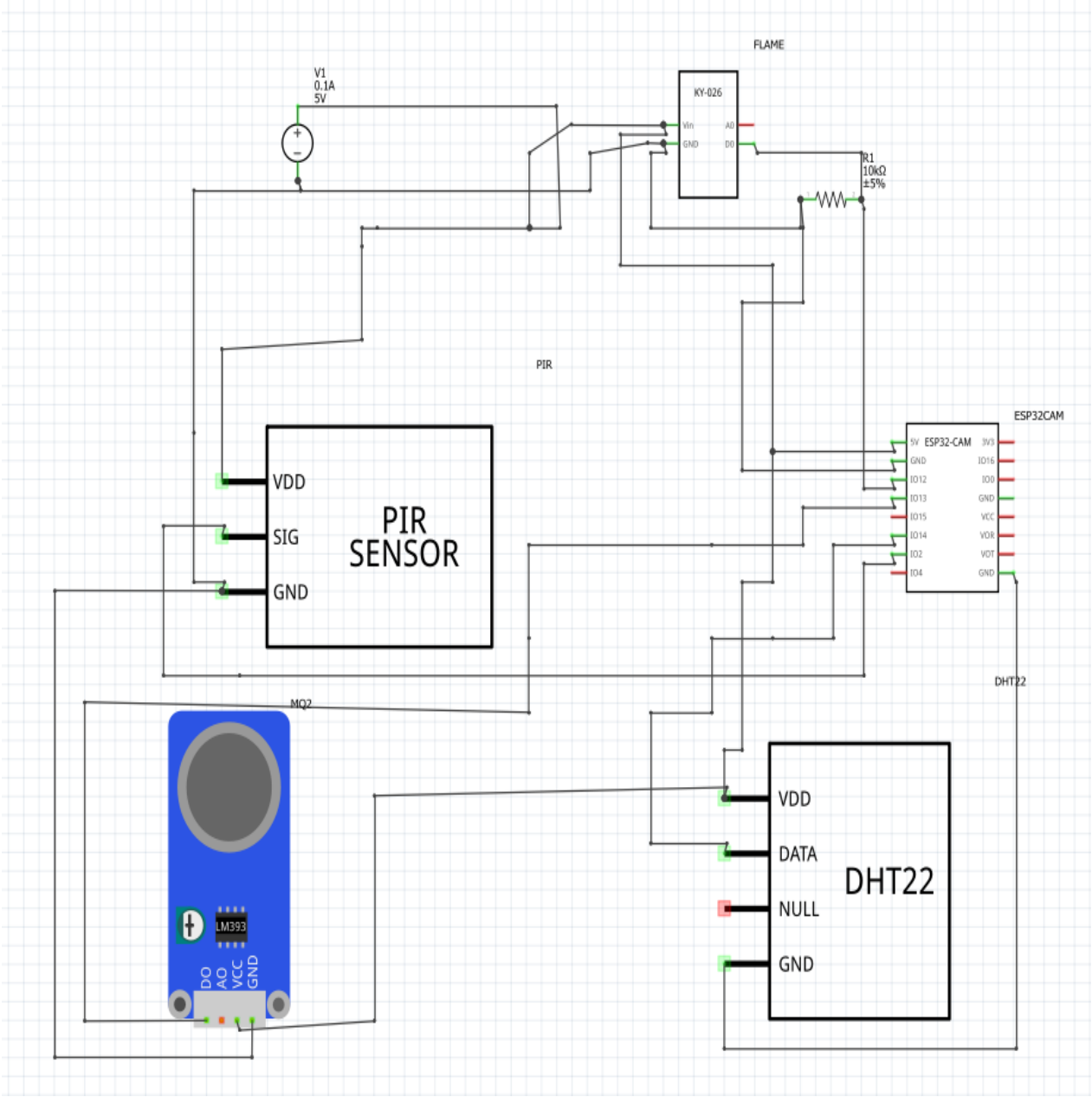

Fig.2

The schematic diagram shows the logical connections of all components. The ESP32-CAM is connected to sensors via GPIO pins (DHT22–GPIO14, MQ2–GPIO13, Flame–GPIO12, PIR–GPIO2). A 10kΩ pull-down resistor is used at GPIO12 for boot stability.

PCB Design

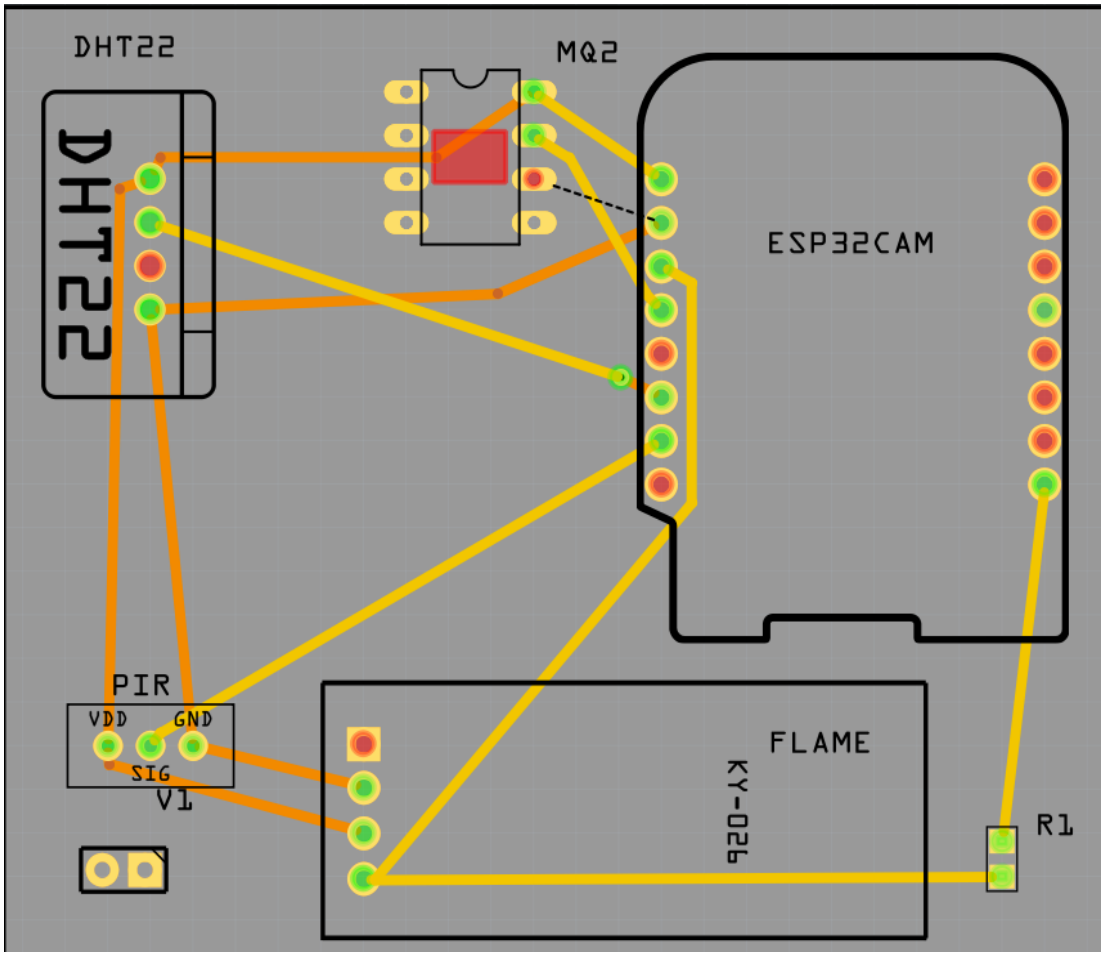

Fig.3

The PCB layout represents the final physical design of the circuit. Components are arranged compactly with proper power and ground routing, improving reliability and reducing loose connections.

Methodology / System Design

The IoT Smart Room Environmental Monitoring and Security System is designed as an integrated safety solution that combines multi-sensor environmental monitoring, motion-based security, camera-based visual verification, and cloud connectivity. The overall methodology is divided into layered architecture consisting of Sensor Layer, Processing & Control Layer (ESP32-CAM), Communication Layer, and Cloud Alert Layer. All layers work together to ensure continuous real-time monitoring and rapid emergency response during hazardous conditions or intrusions.

1. System Architecture

The system follows a modular and layered design where each module performs a specific function. The architecture includes:

- **Sensor Layer** Responsible for acquiring environmental and security data using DHT22, MQ-2, Flame Sensor, and PIR Motion Sensor.
- **Processing & Control Layer (ESP32-CAM based)** Acts as the central unit for data processing, decision making, image capture, and alert triggering.
- **Communication Layer** Handles Wi-Fi connectivity, web server for live streaming, and data exchange with cloud platforms.
- **Cloud & Alert Layer** Consists of Blynk IoT platform for real-time dashboard and Telegram Bot for instant alerts with photos and live stream links.

2. Working Methodology

a) Data Acquisition from Sensors

The ESP32-CAM continuously collects data from the following sensors:

1. **DHT22 Sensor (Temperature & Humidity)** Measures ambient temperature and humidity in real time. It uses a single-wire digital protocol and sends accurate readings to the ESP32-CAM for environmental monitoring.
2. **MQ-2 Gas Sensor** Detects smoke, LPG, and other combustible gases. It provides digital output (LOW when gas concentration exceeds the threshold) and helps in early fire hazard detection.

3. **Flame Sensor (IR-based)** Detects infrared radiation emitted by flames. It gives a digital output (LOW when flame is detected) for immediate fire detection.
4. **PIR Motion Sensor (HC-SR501)** Detects human body movement using infrared radiation. It outputs HIGH signal when motion is detected and is used for intrusion/security monitoring.

b) Data Processing by ESP32-CAM

The ESP32-CAM processes all sensor inputs and evaluates alert conditions. It continuously reads:

- Temperature and humidity from DHT22
- Gas status from MQ-2 (digital)
- Flame status from Flame Sensor (digital)
- Motion status from PIR Sensor

The system checks for any abnormal condition using logical OR operation (gas OR flame OR motion). A 60-second startup delay is implemented to allow sensors (especially MQ-2) to stabilize and avoid false triggers during boot.

An alertActive flag is used along with millis()-based timing to implement debounce logic and prevent repeated alerts for the same event.

c) Camera Activation and Image Capture

When any abnormal condition is detected:

- The built-in OV2640 camera is activated (initialized once in setup() for stability).
- A high-quality image is captured using the ESP32 camera library.
- The captured image is immediately returned to free memory and sent via Telegram.

The system also hosts an internal web server that provides a live MJPEG video stream accessible via the device's local IP address.

d) Cloud Communication and Alert Generation

The ESP32-CAM uses Wi-Fi to connect to the internet. Upon detection of any anomaly:

- Real-time sensor values (temperature, humidity, gas, flame, motion) are continuously sent to the Blynk dashboard using virtual pins (V0 to V4).
- An instant alert is generated through Telegram Bot API. The message includes:
    - Clear text alert ("ALERT DETECTED!")
    - Captured photo of the room
    - Live stream link

The built-in LED on GPIO 33 turns ON during the alert condition for local visual indication.

e) Power Management and Stability

The entire system is powered through a single 5V 2A adapter connected to the 5V pin of the ESP32-CAM. All sensors share the same +5V rail and common GND. A 1000µF electrolytic capacitor is placed across the 5V and GND rails near the ESP32-CAM to prevent brownout conditions during simultaneous Wi-Fi transmission and camera operation. A 10kΩ pull-down resistor on GPIO 12 ensures reliable boot of the ESP32-CAM.

Algorithm / Process Flow

1. The IoT Smart Room Environmental Monitoring and Security System follows a structured sequence of operations to continuously monitor environmental parameters and security threats, detect anomalies, capture visual evidence, and generate instant alerts. The overall process begins with system initialization followed by continuous monitoring of all connected sensors.

2. First, the system initializes the ESP32-CAM, connects to the configured Wi-Fi network, and starts the Blynk service and internal web server. The camera is initialized once in the setup phase for memory stability. A 60-second startup delay is observed to allow the MQ-2 gas sensor to warm up and to prevent false triggers during boot.

3. Next, the system continuously reads the status of all sensors in the main loop. The DHT22 sensor provides temperature and humidity values, while the MQ-2 gas sensor, Flame sensor, and PIR motion sensor provide digital outputs. These values are immediately sent to the Blynk dashboard using virtual pins (V0 for temperature, V1 for humidity, V2 for gas status, V3 for flame status, and V4 for motion status) for real-time remote monitoring.

4. The system then checks for any abnormal condition by evaluating the logical OR of gas detection (MQ-2 output LOW), flame detection (Flame sensor output LOW), or motion detection (PIR output HIGH). If no anomaly is detected, the built-in LED on GPIO 33 remains OFF and the system continues normal monitoring. If any anomaly is detected and the alertActive flag is false, the system activates the alert sequence.

5. Upon anomaly detection, the built-in LED on GPIO 33 turns ON to provide local visual indication. The ESP32-CAM captures an image using the integrated OV2640 camera module. The captured image is immediately sent to the user via Telegram Bot API along with a text alert message containing the live stream link. This provides instant visual confirmation of the event.

6. After sending the alert and photo, the alertActive flag is set to true to implement debounce logic and prevent repeated alerts for the same event. When the abnormal condition clears, the flag is reset, the LED turns OFF, and normal monitoring resumes. The complete cycle runs in a continuous non-blocking loop using millis()-based timing, ensuring

reliable real-time monitoring, power efficiency, and immediate response to environmental hazards or security threats.

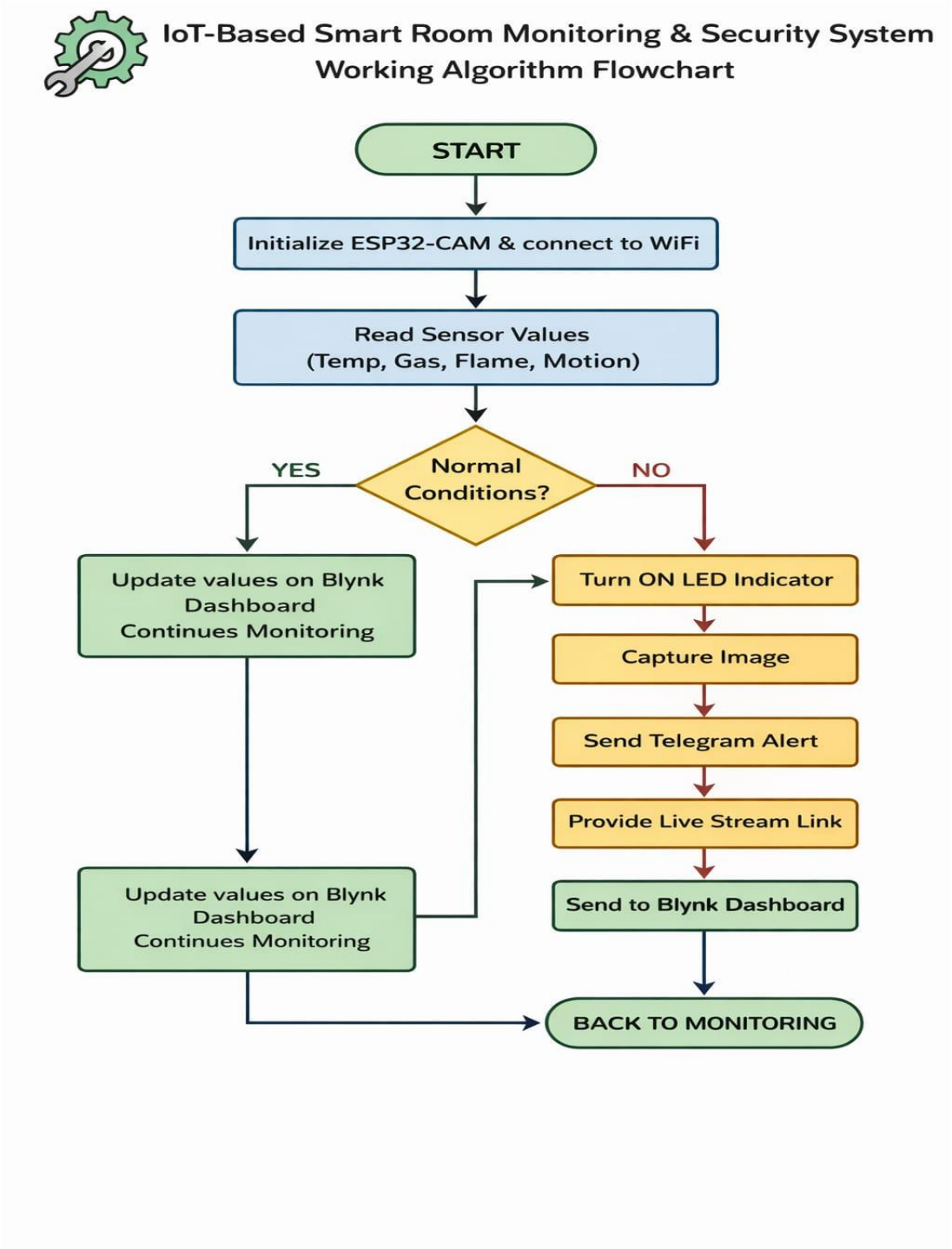

Fig.4

Block Diagram

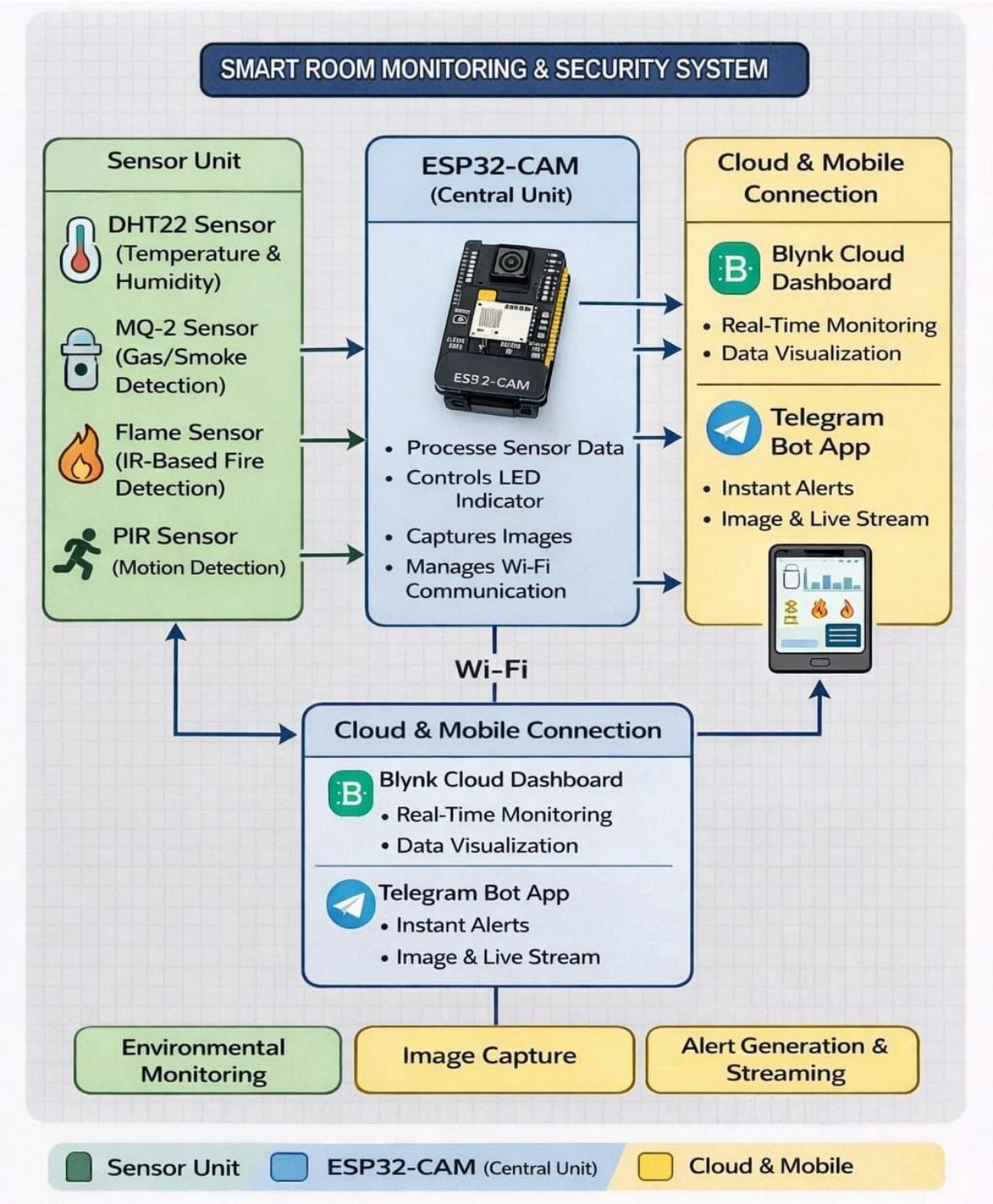

Fig.5

Arduno IDE Code

esp32cam-code.ino

```
/************ BLYNK CONFIG ************/
#define BLYNK_TEMPLATE_ID "TMPL3zOLc4qj9"
#define BLYNK_TEMPLATE_NAME "IOT SMART ROOM MONITOR"
#define BLYNK_AUTH_TOKEN "jFKIL2VlxBV9Jx6qkFwSeBRydz8WjZZ7"

/************ LIBRARIES ************/
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <WiFiClientSecure.h>
#include <esp_camera.h>
#include <WebServer.h>
#include "DHT.h"

/************ WIFI ************/
char ssid[] = "Airtel_Lakshya";
char pass[] = "Lakshya@2005";

/************ TELEGRAM ************/
#define BOT_TOKEN "8260273064:AAGeCSSdZ_H1crOQv8J4jFXnLWfrKiQAXHI"
#define CHAT_ID "5622745127"

/************ SENSOR PINS ************/
#define DHT_PIN 14
#define MQ2_PIN 13
#define FLAME_PIN 12
#define PIR_PIN 2
#define BUILTIN_LED 33

#define DHTTYPE DHT22
DHT dht(DHT_PIN, DHTTYPE);
```

esp32cam-code.ino

```

/************ CAMERA PINS ************/
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

WebServer server(80);

unsigned long startupTime;
bool alertActive = false;

```

esp32cam-code.ino

```
/*********** CAMERA INIT ***********/
void initCamera() {

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;

  if (psramFound()) {
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 12;
    config.fb_count = 2;
  } else {
    config.frame_size = FRAMESIZE_QVGA;
    config.jpeg_quality = 15;
    config.fb_count = 1;
  }

  esp_camera_init(&config);
}
```

esp32cam-code.ino

```
92
93  /************ STREAM SERVER ************/
94  void handleRoot() {
95    server.send(200, "text/html",
96                "<html><body><h2>Live Stream</h2><img src=\"/stream\"></body></html>");
97  }
98
99  void handleStream() {
100   WiFiClient client = server.client();
101   String response = "HTTP/1.1 200 OK\r\n";
102   response += "Content-Type: multipart/x-mixed-replace; boundary=frame\r\n\r\n";
103   server.sendContent(response);
104
105   while (client.connected()) {
106     camera_fb_t * fb = esp_camera_fb_get();
107     if (!fb) break;
108
109     server.sendContent("--frame\r\n");
110     server.sendContent("Content-Type: image/jpeg\r\n\r\n");
111     server.sendContent((char*)fb->buf, fb->len);
112     server.sendContent("\r\n");
113     esp_camera_fb_return(fb);
114   }
115 }
116
```

esp32cam-code.ino

```
116
117 /************ TELEGRAM TEXT (POST FIXED) ************/
118 void sendTelegramMessage(String message) {
119
120   WiFiClientSecure client;
121   client.setInsecure();
122
123   if (client.connect("api.telegram.org", 443)) {
124
125     String payload = "chat_id=" + String(CHAT_ID) +
126                      "&text=" + message;
127
128     client.println("POST /bot" + String(BOT_TOKEN) + "/sendMessage HTTP/1.1");
129     client.println("Host: api.telegram.org");
130     client.println("Content-Type: application/x-www-form-urlencoded");
131     client.print("Content-Length: ");
132     client.println(payload.length());
133     client.println();
134     client.print(payload);
135   }
136 }
137
```

esp32cam-code.ino

```
137
138 /************ TELEGRAM PHOTO ************/
139 void sendPhotoTelegram() {
140
141   camera_fb_t * fb = esp_camera_fb_get();
142   if (!fb) return;
143
144   String boundary = "ESP32";
145   String head = "--" + boundary + "\r\nContent-Disposition: form-data; name=\"chat_id\";\r\n\r\n" + String(CHAT_ID) + "\r\n";
146   head += "--" + boundary + "\r\nContent-Disposition: form-data; name=\"photo\"; filename=\"image.jpg\"\r\nContent-Type: image/jpeg\r\n\r\n";
147   String tail = "\r\n--" + boundary + "--\r\n";
148
149   WiFiClientSecure client;
150   client.setInsecure();
151
152   if (client.connect("api.telegram.org", 443)) {
153     client.println("POST /bot" + String(BOT_TOKEN) + "/sendPhoto HTTP/1.1");
154     client.println("Host: api.telegram.org");
155     client.println("Content-Type: multipart/form-data; boundary=" + boundary);
156     client.print("Content-Length: ");
157     client.println(head.length() + fb->len + tail.length());
158     client.println();
159     client.print(head);
160     client.write(fb->buf, fb->len);
161     client.print(tail);
162   }
163
164   esp_camera_fb_return(fb);
165 }
```

esp32cam-code.ino

```
166
167 /************ SETUP ************/
168 void setup() {
169
170   Serial.begin(115200);
171
172   pinMode(MQ2_PIN, INPUT);
173   pinMode(FLAME_PIN, INPUT);
174   pinMode(PIR_PIN, INPUT);
175   pinMode(BUILTIN_LED, OUTPUT);
176   digitalWrite(BUILTIN_LED, HIGH);
177
178   dht.begin();
179
180   WiFi.begin(ssid, pass);
181   while (WiFi.status() != WL_CONNECTED) delay(500);
182
183   Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
184
185   initCamera();
186
187   server.on("/", handleRoot);
188   server.on("/stream", HTTP_GET, handleStream);
189   server.begin();
190
191   startupTime = millis();
192 }
```

esp32cam-code.ino

```
192  }
193
194  /************ LOOP ************/
195  void loop() {
196
197    Blynk.run();
198    server.handleClient();
199
200    float temp = dht.readTemperature();
201    float hum = dht.readHumidity();
202
203    if (!isnan(temp)) Blynk.virtualWrite(V0, temp);
204    if (!isnan(hum)) Blynk.virtualWrite(V1, hum);
205
206    bool gas = digitalRead(MQ2_PIN) == LOW;
207    bool flame = digitalRead(FLAME_PIN) == LOW;
208    bool motion = digitalRead(PIR_PIN) == HIGH;
209
210    Blynk.virtualWrite(V2, gas);
211    Blynk.virtualWrite(V3, flame);
212    Blynk.virtualWrite(V4, motion);
213
214    bool anyAlert = gas || flame || motion;
215
216    if (millis() - startupTime > 60000) {
217
218      if (anyAlert && !alertActive) {
219
220        alertActive = true;
221        digitalWrite(BUILTIN_LED, LOW);
222
223        String ip = WiFi.localIP().toString();
224        String message = "ALERT DETECTED!\nLive Stream:\nhttp://" + ip;
225
226        sendTelegramMessage(message);
227        sendPhotoTelegram();
228      }
229
230      if (!anyAlert) {
231        alertActive = false;
232        digitalWrite(BUILTIN_LED, HIGH);
233      }
234    }
235  }
236
237
```

Blynk IoT Dashboard

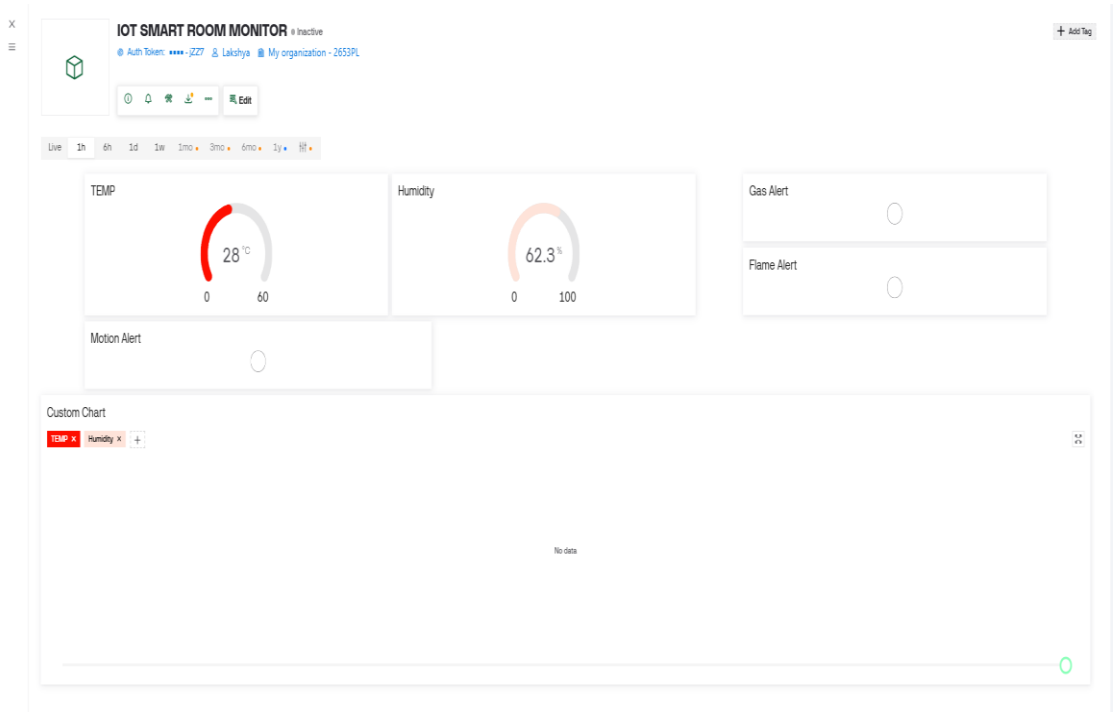

Fig.6

Telegram Bot Interface

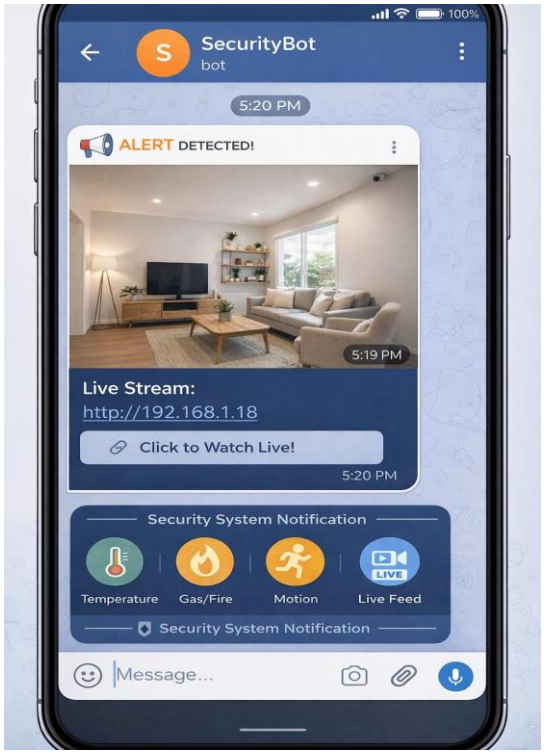

Fig.7

Results and Testing

Extensive testing was carried out on the IoT Smart Room Environmental Monitoring and Security System using ESP32-CAM to verify the functionality of each module and the overall performance of the integrated setup. The results demonstrate that the system works reliably under normal and stress conditions and successfully performs all intended environmental monitoring and security functions.

1. Sensor Reading and Blynk Dashboard Test • The DHT22, MQ-2, Flame sensor, and PIR sensor were tested by exposing them to normal room conditions and simulated anomalies. • Result: All sensors provided accurate digital readings. Temperature and humidity values from DHT22, gas status from MQ-2, flame status from Flame sensor, and motion status from PIR were successfully displayed on the Blynk dashboard in real time using virtual pins V0 to V4. Historical data graphs were also generated correctly.

2. Gas and Smoke Detection Test • Different levels of smoke and combustible gas were simulated near the MQ-2 sensor by introducing smoke from incense or lighter fluid (in a controlled environment). • Result: The MQ-2 sensor consistently detected gas/smoke and triggered the alert condition. The system sent instant Telegram alerts with captured photo and live stream link. No false triggers were observed after the 60-second warm-up period.

3. Flame Detection Test • A small controlled flame (using a lighter at safe distance) was brought near the Flame sensor multiple times. • Result: The Flame sensor reliably detected the infrared radiation and activated the alert sequence. The built-in LED turned ON, image was captured, and Telegram notification with photo was delivered within 2–3 seconds. The 10kΩ pull-down resistor on GPIO 12 ensured stable boot without issues.

4. Motion Detection Test (PIR Sensor) • Human movement was simulated in front of the PIR sensor at various distances and angles. The PIR jumper was set to “H” (retrigger) mode. • Result: The PIR sensor accurately detected motion and triggered alerts only after the debounce logic (alertActive flag). False triggers due to air movement were minimized. Live stream link was successfully shared via Telegram for visual confirmation.

5. Camera and Image Capture Test • The OV2640 camera was tested under different lighting conditions (normal room light and low light). • Result: The camera initialized successfully once in setup() and captured clear images during alert conditions. Photos were reliably sent via Telegram Bot API. The internal web server provided smooth MJPEG live streaming at the device's local IP address without crashing.

6. Power Stability and Boot Safety Test • The system was power-cycled more than 30 times with and without the 1000µF capacitor. Multiple sensors were connected and disconnected during testing. • Result: With the 1000µF electrolytic capacitor placed across the 5V and GND rails and the 10kΩ pull-down on GPIO 12, the ESP32-CAM booted reliably every time without brownout or bootloop issues. Wi-Fi reconnection worked smoothly after network drops.

7. Telegram Alert and Live Stream Test • Multiple alert conditions (gas, flame, motion) were triggered intentionally. • Result: Instant alerts containing text message, captured photo, and live stream link were successfully delivered to the Telegram chat. Response time was observed to be 2–5 seconds depending on Wi-Fi speed. The alertActive flag effectively prevented alert flooding.

8. Overall System Stress Test • The complete system was run continuously for more than 3 hours under repeated trigger conditions with Wi-Fi on/off cycles. • Result: No system crash, memory leak, or unexpected reset was observed. Heap memory remained stable, and the system maintained reliable performance throughout the testing period.

Future Scope / Enhancements

The IoT Smart Room Environmental Monitoring and Security System developed in this project provides essential environmental monitoring and security features; however, there are several opportunities for further improvement and expansion. These enhancements can significantly increase the overall reliability, efficiency, and user experience of the system.

1. Integration of IoT Cloud Platforms Future versions can directly upload sensor data, images, and alert logs to cloud platforms such as Firebase, ThingSpeak, or AWS IoT. This would allow real-time tracking, remote monitoring, long-term data storage, and advanced analytics.
2. **Addition of Air Quality Sensor (MQ-135 or PMS5003) Integrating a dedicated air quality sensor would enable monitoring of PM2.5, CO_2,** and other pollutants, making the system more comprehensive for smart home and laboratory environments.
3. **SIM-Based GSM/GPRS Emergency Notification** The system can be expanded to include automatic SMS or voice calling to emergency numbers using a GSM module, ensuring alerts are delivered even when Wi-Fi or internet is unavailable.
4. Advanced Camera Features with **AI Machine learning models** (TensorFlow Lite) can be deployed on ESP32-CAM or a companion board to perform object detection, face recognition, or fire classification directly on the edge for faster and more intelligent alerts.
5. **Mobile App Development A dedicated Android/iOS** application can be developed instead of relying only on Blynk and Telegram. The app can provide push notifications, live video streaming, historical data visualization, and manual camera control.
6. **Solar-Powered or Battery Backup System** Small solar panels or Li-ion battery with charging circuit can be added to make the system independent of continuous mains power, ideal for remote locations or during power outages.
7. **Multi-Room or Multi-Device Synchronization Multiple ESP32-CAM units** can be deployed across different rooms and synchronized using MQTT protocol for centralized monitoring and control from a single dashboard.

8. **Voice Alert and Integration with Smart Assistants Integration with Amazon Alexa**, Google Assistant, or local voice output can enable audible alerts inside the room and voice-controlled commands for checking status or activating the camera.
9. **Data Logging on SD Card or Cloud Adding microSD card support** (with careful GPIO management) or cloud-based logging would allow storing sensor readings and captured images for post-event analysis and auditing.
10. Enhanced Security with **Encryption and Authentication** Implementing secure boot, encrypted communication **(HTTPS/MQTTS**), and user authentication for the web server and Telegram integration would improve data privacy and protect against unauthorized access.

Conclusion

The IoT Smart Room Environmental Monitoring and Security System developed in this project successfully addresses major indoor safety concerns such as fire outbreaks, gas leakage, smoke detection, and unauthorized intrusions. By integrating multiple sensors (DHT22, MQ-2, Flame sensor, and PIR) with the ESP32-CAM module, the system provides real-time environmental monitoring, motion-based security, automatic image capture, and instant cloud-based alerts.

The combination of edge processing on the ESP32-CAM, Blynk dashboard for continuous data visualization, and Telegram Bot for delivering alerts with captured photos and live stream links ensures rapid detection and response to hazardous conditions. The careful design considerations including boot-safe GPIO configuration (10kΩ pull-down on GPIO 12), power stabilization using a 1000µF capacitor, and debounce logic have resulted in a stable and reliable system.

Testing results confirm that the system performs reliably under real-world conditions, offering consistent sensor accuracy, fast alert delivery, smooth camera operation, and effective Wi-Fi reconnection. The design is low-cost, compact, easy to assemble on a breadboard, and can be installed in homes, hostels, offices, or laboratories without requiring complex infrastructure.

Overall, the IoT Smart Room Monitoring project demonstrates an effective blend of environmental sensing, security monitoring, visual verification, and IoT connectivity. It serves as a promising step toward smarter and safer indoor environments and provides a strong foundation for future advancements in intelligent monitoring and safety systems.

This project proves that a low-cost, reliable, and efficient smart room safety system can be developed using simple electronic components and open-source programming. It not only minimizes potential damage by enabling early detection and rapid response but also serves as a scalable solution for advanced safety systems in homes, offices, and industries.

References

1. Bagaskara, A. R. (2025). Design of an Internet of Things (IoT) Based Security System Using ESP32-CAM and Passive Infrared Receiver (PIR) Motion Sensor in the Home Environment. *International Journal of Engineering Computing and Advanced Research*.
2. Wicaksono, M. F., Rahmatya, M. D., & Faturohman, A. C. (2025). Smart Home Monitoring and Control with Human Detection and Telegram Messages using ESP32-Wrover CAM. *Proceedings of the 8th International Conference on Informatics, Engineering, Science & Technology (INCITEST 2025)*.
3. Purba, J. (2026). ESP32 Decision Tree-Based Multi-Hazard Detection System for Smart Safety Room. *Journal of Artificial Intelligence and Engineering Applications*.
4. Esparza-Segundo, F., Enríquez-Arcadio, S. L., Gonzalez-Islas, J. C., Suarez-Navarrete, A., & et al. (2025). ESP32-CAM-based Smart Surveillance: Real-time Transmission and Alerts Via Telegram. *Programación Matemática y Software*.
5. DHT22 Digital-output Relative Humidity & Temperature Sensor Datasheet. Aosong Electronics. Available online at: https://cdn-shop.adafruit.com/datasheets/DHT22.pdf
6. MQ-2 Gas Sensor Module Datasheet. Hanwei Electronics. Available online at: https://www.winsen-sensor.com
7. ESP32-CAM Hardware Reference and Camera Library Documentation. Espressif Systems. Retrieved from: https://www.espressif.com
8. Blynk IoT Platform Documentation for ESP32. Blynk Inc. Available at: https://docs.blynk.io
9. Universal Telegram Bot Library for Arduino/ESP32. Retrieved from: https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot
10. Rahman, M. M., & Islam, M. N. (2021). An Efficient Motion Detection-based Home Security System using ESP32-CAM and Blynk. *International Journal of Computer Applications Technology and Research*, 10(4), 141–146.